Chordy: A Distributed Hash Table

Rodrigo Perestrelo, Lennard Last
ID2201 Distributed Systems, Basic Course

October 7, 2026

1 Introduction

The objective of this assignment is to design, implement, and empirically evaluate **Chordy**, a structured Distributed Hash Table based on the Chord protocol scheme. The system organizes autonomous Erlang processes into an overlay identifier ring where keys and nodes share a uniform discrete space.

2 Iterative Architectural Progression

We developed the system through two foundational stages, transitioning from topological ring maintenance to a fully functioning distributed storage engine.

2.1 Iteration 1: Topology and Self-Stabilizing Ring (`node1`)

The baseline architecture, implemented in modules `key` and `node1`, maintains a circular overlay network without data storage.

- **Circular Identifier Space:** Nodes generate 30-bit random identifiers (1 to 1 000 000 000). The predicate `key:between(Key, From, To)` resolves circular intervals $(From, To]$ by handling standard boundaries ($From < To$), wraparounds ($From > To$), and the full-circle degenerate identity ($From == To$).
- **Asynchronous State Maintenance:** Each node tracks its current identifier, a predecessor pointer, and a successor pointer as $\{Key, Pid\}$ tuples. Ring integrity is maintained asynchronously via four core message primitives:
  - `{key, Qref, Peer}`: Resolves the key of a bootstrap node during connection.
  - `{request, Peer}`: Solicits predecessor information from the successor.
  - `{status, Pred}`: Transmits predecessor status to the querying predecessor.
  - `{notify, New}`: Proposes a friendly candidate as a potential new predecessor.
- **Periodic Stabilization:** Driven by `timer:send_interval/3`, each node queries its successor's predecessor via `request`. If an intermediate node X is discovered such that $X \in (Id, Successor]$, the querying node adopts X as its successor. It then informs the successor of its presence via `notify`.

```
stabilize(Pred, Id, Successor) ->
    {Skey, Spid} = Successor,
    case Pred of
        nil ->
            Spid ! {notify, {Id, self()}},
            Successor;
```

```
        {Id, _} -> Successor;
        {Skey, _} ->
            Spid ! {notify, {Id, self()}},
            Successor;
        {Xkey, Xpid} ->
            case key:between(Xkey, Id, Skey) of
                true ->
                    Xpid ! {request, self()},
                    {Xkey, Xpid};
                false ->
                    Spid ! {notify, {Id, self()}},
                    Successor
            end
    end.
```

Structural Insight: When a node points to itself ($Predecessor == Id$), the condition $From == To$ triggers true in `key:between/3`, ensuring that incoming nodes are accepted into single-node rings without explicit corner-case branches.

2.2 Iteration 2: Distributed Store and Dynamic Handover (`node2`)

Module `node2` integrates a local key-value repository, delegating key-range ownership across ring participants:

- **Responsibility Invariant:** A node Id with predecessor $Pkey$ owns the half-open interval $(Pkey, Id]$. Operations on keys falling outside this interval are forwarded sequentially to the immediate successor `Spid`.
- **Transparent Client Routing:** Client queries (`{add, Key, Value, Qref, Client}` and `{lookup, Key, Qref, Client}`) carry a unique tag `Qref`. When forwarded along the ring, the owning node sends the completion response directly to `Client`, eliminating intermediate return hops.
- **Dynamic Storage Delegation (`handover`):** When a new predecessor $Nkey$ joins, the interval $(Pkey, Id]$ is partitioned. The local node retains $(Nkey, Id]$ and delegates $(Pkey, Nkey]$ by dispatching a `{handover, Rest}` message to $Npid$.

```
handover(Id, Store, Nkey, Npid) ->
    {Keep, Rest} = storage:split(Nkey, Id, Store),
    Npid ! {handover, Rest},
    Keep.
```

3 Empirical Evaluation

3.1 Evaluation 1: Single-Node and Multi-Node Probe Verification

We initialized a distributed ring across separate Erlang VM nodes to measure ring transit latency and verify pointer convergence using a circumnavigating probe message:

```
(first@127.0.0.1)1> P1 = node1:start(100).
<0.94.0>
(first@127.0.0.1)2> register(chord_root, P1).
true
(first@127.0.0.1)3> chord_root !  probe.
probe
Probe returned in 419 microseconds after 3 hops:  [<10738.94.0>, <10737.94.0>, <0.94.0>]
```

Observed Result: The probe completed a full circle across 3 independent OS-level Erlang processes in 419 microseconds, confirming that periodic stabilization converged into a consistent, closed ring topology.

3.2 Evaluation 2: Scalability and Performance Profiling

We executed sequential and concurrent benchmark routines to evaluate the system under varying ring topologies ($N = 1$, $N = 2$, and $N = 4$ nodes) storing 1 000 elements:

```
8> benchmark:benchmark(1000, N1).  Starting addition of 1000 elements...  Addition
completed in 6361 us (6 ms) Starting lookup of 1000 elements...  Lookup completed in 1818
us (1 ms) 6361, 1818
11> benchmark:benchmark(1000, N1).  Starting addition of 1000 elements...  Addition
completed in 9713 us (9 ms) Starting lookup of 1000 elements...  Lookup completed in 2430
us (2 ms) 9713, 2430
15> benchmark:benchmark(1000, N1).  Starting addition of 1000 elements...  Addition
completed in 8413 us (8 ms) Starting lookup of 1000 elements...  Lookup completed in 2697
us (2 ms) 8413, 2697
16> benchmark:benchmark(1000, N1, 4).  Starting benchmark with 4 clients (250 keys each)...
Parallel benchmark completed in 12469 us (12 ms) 12469
```

Configuration	Add Time (μs)	Lookup Time (μs)	Avg Add/Op	Avg Lookup/Op
1 Node (1 000 keys)	6 361	1 818	6.36 μs	1.82 μs
2 Nodes (1 000 keys)	9 713	2 430	9.71 μs	2.43 μs
4 Nodes (1 000 keys)	8 413	2 697	8.41 μs	2.70 μs
4 Concurrent Clients	12 469 μs (Total Pipeline)		Contention on N_1 Mailbox	

Table 1: Empirical benchmark timings across ring configurations.

3.3 Evaluation 3: Architectural Analysis and Limiting Factors

- **Single-Node vs Multi-Node Overhead:** In a single-node configuration, 100% of requests are local, eliminating inter-process message passing. When scaling to 2 nodes, approximately 50% of operations require forwarding to the successor, increasing addition latency from 6 361 μs to 9 713 μs.
- **Local Complexity Mitigation ($N = 4$):** Expanding the ring to 4 nodes reduced addition latency to 8 413 μs. Because each node stores approximately 250 keys instead of 1 000, the $O(K)$ traversal cost of `lists:keystore/4` drops significantly, offsetting message routing overhead.
- **Successor-Hop Forwarding Penalty:** Lookup latency rose monotonically (1 818 μs $\rightarrow$ 2 430 μs $\rightarrow$ 2 697 μs). Because our ring routes via immediate successors ($O(N)$ hops) without finger routing tables ($O(\log N)$), each additional node lengthens the traversal path.
- **The Mailbox Bottleneck:** When 4 client processes submit requests concurrently through a single entrypoint (N_1), latency climbs to 12 469 μs. The entry node's mailbox serializes all inbound traffic, creating an architectural bottleneck.

4 Handling of Node Failure

In the third iteration of **Chordy**, the handling of failures and subsequent self-healing was added. The implementation has the following three key additions.

4.1 Neighbor Monitoring

Each node now monitors its neighbors using Erlang's built-in `monitor` mechanism. This way a node will be notified when its predecessor or successor fails by receiving a `{'DOWN', Ref, ..}` message and can react accordingly.

4.2 Next Pointer

During stabilization a node will now also pass its own successor in the request reply and the recipient of the status message will save it in the `Next` variable of the node. This pointer is needed because a node that loses its successor would otherwise have no way to reach the rest of the ring. When a closer successor is found during stabilization, the old successor becomes the new `Next`.

```
request(Peer, Predecessor, {Skey, _, Spid}) ->
    case Predecessor of
        nil ->
            Peer ! {status, nil, {Skey, Spid}};
        {Pkey, _, Ppid} ->
            Peer ! {status, {Pkey, Ppid}, {Skey, Spid}}
    end.
```

```
node(Id, Predecessor, Successor, Next, Store) ->
    receive
        ...
        {status, Pred, Nx} ->
            {Succ, Nxt} = stabilize(Pred, Nx, Id, Successor),
            node(Id, Predecessor, Succ, Nxt, Store);
```

4.3 Failure Handling

When the successor fails, the node will set `Next` as its new successor and trigger stabilization to inform its new successor and receive a new `Next` pointer. When the predecessor fails, the node will just set it to `nil` and wait for the new predecessor's stabilization.

```
down(Ref, {_, Ref, _}, Successor, Next) ->
    %% Predecessor died
    {nil, Successor, Next};
down(Ref, Predecessor, {_, Ref, _}, {Nkey, Npid}) ->
    %% Successor died
    drop(Ref),
    Nref = monitor(Npid),
    self() ! stabilize,
    {Predecessor, {Nkey, Nref, Npid}, nil}.
```

5 Replication

The self-healing capabilities in the third iteration may keep the ring up and running, but a node that fails takes all its data with it. To counter this a simple form of replication is implemented in the fourth iteration.

Now every node will store a `Replica` of its predecessor's store. This is a sensible design decision, since upon exit of a node, its successor is now responsible for its data and thus can just move `Replica` into its own `Store`.

```
down(Ref, {_, Ref, _}, Successor, Next, Store, Replica) ->
    %% Predecessor died
    MergedStore = storage:merge(Store, Replica),
```

```
    NewReplica = storage:create(),
    {nil, Successor, Next, MergedStore, NewReplica};
```

When a node adds a key to its own store, it forwards the pair to its successor in a `replicate` message. The successor stores it in its `Replica` and sends the confirmation to the client, so the client only receives `ok` once both copies exist.

```
{replicate, Key, Value, Qref, Client} ->
    NewReplica = storage:add(Key, Value, Replica),
    Client ! {Qref, ok},
    node(Id, Predecessor, Successor, Next, Store, NewReplica);
```

To make sure replication stays consistent through changes of the ring, the `handover` function now also returns the part of the storage that is the responsibility of the new predecessor and the `notify` function then returns it to be set as the new `Replica`.

```
handover(Id, Store, Nkey, Npid) ->
    {Keep, Rest} = storage:split(Nkey, Id, Store),
    Npid ! {handover, Rest},
    {Keep, Rest}.
```

```
{notify, New} ->
    case notify(New, Id, Predecessor, Store) of
        {Pred, NewStore, nil} ->
            node(Id, Pred, Successor, Next, NewStore, Replica);
        {Pred, NewStore, NewReplica} ->
            node(Id, Pred, Successor, Next, NewStore, NewReplica)
    end;
```

6 Conclusions

This laboratory demonstrates the mechanics of structured decentralized storage over an identifier ring. Decentralized ring stabilization allows autonomous nodes to dynamically self-organize without a centralized coordinator. While distributed partitioning divides local storage costs, linear successor routing incurs an $O(N)$ hop penalty, and single-entry client access introduces mailbox serialization. Achieving high-throughput scalability requires finger-table routing and multi-point client dispatching.